[Final Report Template]

[Block Y1B-2024-25]

Important Notes:

- Please, rename the file to FinalReport_<your_name>_<studentnumber>.**PDF** before uploading it.
- Upload this template to the 'Deliverables' folder in your BUas GitHub repository.
- You can add as many sections/ subsections as needed.
- You should structure sections based on paragraphs (you can add bullet points, numberings, and plots if needed).
- The final report should be submitted in **"PDF"** format only.

Introduction

You determine your business case by understanding the dataset and the features in it that you predict, classify or cluster based on machine learning models.

Problem Statement

A problem overview explaining your implemented baselines to examine problems that can be addressed by regression or classification or clustering.

This project focuses on analyzing and modeling a healthcare dataset to address key challenges and gain meaningful insights using machine learning techniques. The analysis incorporates both supervised and unsupervised learning methods to explore three distinct problem types: regression, classification, and clustering. Each problem type was approached with specific goals and carefully chosen models to solve the associated tasks.

For regression, the goal was to predict continuous outcomes, specifically the length of hospital stays represented by the variable time_in_hospital. This involved identifying numerical and categorical features that were likely to influence the target variable. Two regression models were implemented. The first was a Linear Regression model, which served as a baseline. Linear Regression was chosen for its simplicity and interpretability, making it easier to understand the relationships between features and the target variable. Features such as num_medications, num_lab_procedures, and discharge_disposition_id were selected because they directly reflect the complexity of the patient's medical treatment. The second model was a Random Forest Regressor, which was implemented to handle potential nonlinear relationships in the data. Unlike Linear Regression, Random Forest can

capture more complex patterns and also provides insights into feature importance, offering a deeper understanding of which variables have the most significant impact on hospital stay length.

For classification, the objective was to categorize patients based on specific outcomes, such as whether or not they were readmitted to the hospital. This task involved both numerical and categorical features that could influence readmission likelihood. A RandomForestClassifier was used for this problem, as it is a robust model capable of handling mixed data types and class imbalances. The model's performance was evaluated using the F1-score, a metric that considers both precision and recall, making it well-suited for imbalanced datasets. Features like admission_type_id, discharge_disposition_id, and num_medications were used to train the model because they provided meaningful information about patient admissions and treatment complexity, which are critical factors in determining readmission risk.

For clustering, the goal was to group patients into distinct clusters based on patterns in their medical data. This unsupervised learning task aimed to uncover underlying structures in the dataset, such as commonalities in laboratory procedures or medication usage. KMeans Clustering was applied as the primary algorithm, grouping patients based on similarities in selected numerical features. Standardized features, such as num_medications, num_lab_procedures, and number_inpatient, were used to ensure all variables contributed equally to the clustering process. The performance of the clustering model was evaluated using the Silhouette Score, which measures how well-separated and cohesive the clusters are. While the clustering results were not as strong as expected, the process provided valuable insights into the dataset's distribution and potential areas for further refinement.

Overall, this project involved a comprehensive application of machine learning techniques to address regression, classification, and clustering problems. Each model was carefully selected and tailored to the specific goals of the task, ensuring meaningful results and actionable insights from the data.

1) Exploratory Data Analysis (EDA)

SQL Part

The first step in EDA is to gain a general understanding of the dataset. For instance, to determine the total number of records in a specific column, such as encounter_id, we can write a query to calculate the total rows in that column. This helps us understand the overall size of the data and check for its completeness, ensuring that no key information is missing at the foundational level.

Next, we examined the distribution of different admission types. By analyzing this information, we gained insights into the variety of reasons patients were admitted to the hospital, which is crucial for identifying patterns in patient demographics or medical needs.

Another important aspect of the analysis involved exploring discharge dispositions. Using a query, we calculated how often each discharge disposition appeared in the dataset. The results were grouped by discharge type, sorted from most to least frequent, and the top 10 results were displayed. This analysis provided a clearer picture of where patients typically went after being discharged, such as home, nursing facilities, or other medical institutions.

To ensure data integrity, it was necessary to identify missing or anomalous data. For example, we checked the race column for missing values and calculated the occurrence of each value in the column. This helped us identify whether any significant gaps existed in the data and how evenly the data was distributed among different categories. Similarly, we investigated the weight column for missing or unusual values. In SQL, missing values are typically represented as NULL. When we checked for null values in the weight column, the result was zero, indicating that the column had no missing values.

The next step was to calculate the age distribution within the dataset. By querying the minimum and maximum values in the age column, we established the overall age range of patients in the dataset. We also calculated the number of patients in each age group to better understand the demographic breakdown. This analysis provided critical insights into the age groups most commonly represented in hospital admissions, which could influence further modeling decisions.

We then analyzed the contribution of different admission sources to hospital admissions. This task highlighted which sources—such as emergency rooms, physician referrals, or transfers from other facilities—were most commonly associated with patient admissions. Understanding this information is important for identifying trends in how patients enter the hospital system.

Another key analysis was determining the average time spent in the hospital for each admission type. Using SQL, we grouped the data by admission type and calculated the average length of hospital stay. The results were then sorted to display admission types with the longest stays first

EDA with python

To start exploratory data analysis, first, we need to import our dataset, and we do this with the help of pandas. But before using pandas, we need to make sure it is imported into our environment. After that, we can load the data, usually with functions like pd.read_csv().

Once we have the data in our Jupyter Notebook, the next step is to check the shape of the dataset. We need to see how many rows and columns there are, which we can find using .shape. We can also check the names of the features (columns) using .columns and get detailed information about the dataset using .info. This will give us a good idea about the size of the data, the types of each column, and whether we have missing values.

After inspecting the structure, we need to handle missing values. It is important to remove or deal with NaN (missing) values because if we run our code with them, the results might not be accurate, and our machine learning model could get confused and perform poorly. Removing or imputing NaN values is a crucial step to ensure the data is clean.

Next, we should check the unique values in each column. This helps us identify if there are any redundant or irrelevant values. For example, some values might need to be merged, or certain symbols might not be meaningful. Cleaning these values is essential. We should also look for duplicate rows or columns and remove them because duplicates can lead to overfitting and make our model less reliable. If we feed too much redundant or unnecessary data to the model, it might only memorize the training data instead of learning to generalize and predict correctly.

Finally, to better understand our data, we should use visualization. Visualizations make patterns and insights more clear and easier to understand without needing too much time. For instance, I used matplotlib to visualize features like age, race, gender, or other meaningful data to check if there are any remaining missing values or anomalies. While visualizing, it's also a good idea to check unique values to ensure that no NaN or missing values are left. This step makes the data more reliable and ready for further analysis or modeling.

2) Data Processing

To start data processing, first, you need to have some important libraries in your environment like pandas, numpy, matplotlib, and seaborn. These libraries are essential because they help you handle, clean, and visualize the data properly, which is the most important step in data preprocessing.

```
# Importing required Python libraries
import pandas as pd
import numpy as ns
import seaborn as sns
import matplotlib.pyplot as plt

# Loading the data
path=r"C:\Users\Asus\Desktop\diabetic_data.csv"

file=pd.read_csv(path)
```

The reason we use visualization tools like seaborn and matplotlib is that they allow us to get a complete view of the dataset. Visualization makes it easy to understand where the missing or NaN values are located and also helps identify the unique values in each column. This is important because sometimes, we need to merge or remove unnecessary values that don't add meaning to our dataset. For example, if a column has repetitive or irrelevant symbols, we need to clean it up.

Another useful step in data processing is splitting the dataset into two separate datasets: categorical data and numerical data. This is important because some machine learning models only work with numerical data. Once you separate the data, it becomes easier to handle. For instance, categorical data can be converted into numerical values using encoding techniques like one-hot encoding or label encoding.

```
# Display data types of each column
print(file.dtypes)

for column in file.select_dtypes(include=['object']).columns:
    print(column)
```

Handling duplicate rows is another important step. Duplicate rows increase the size of the dataset unnecessarily, making it harder for the machine learning model to process and learn efficiently. By removing duplicates, you can reduce the dataset size, which will also save time during computation and improve the model's performance. The smaller and cleaner the dataset, the faster and more accurate the model will be.

```
for i in file.duplicated():
    count=0
    if i==True:
        count+=1



print(f"Number of duplicate rows: {count}")

# Drop duplicates if any
x=file.drop_duplicates()
print(x)
```

When dealing with missing values, it is important to decide how to handle them:

For numerical features, you can replace missing values with the mean, which is simple and ensures that the overall distribution of the data remains consistent.

For categorical features, you can fill missing values with the most frequent category to preserve the dataset's structure and avoid introducing randomness. If a column has too many missing values, like more than 30-40%, it’s often better to drop it because filling so many missing values could reduce the dataset’s reliability and add noise.

```
# Identify and calculate the percentage of missing values
missing=file.isnull().sum()
print(missing)
rows=file.shape[0]
percentage=(missing/rows)*100
print(percentage)
print("Missing values (in percentage) for each column:\n",  )
```

Another key step in data processing is understanding the metadata of your dataset. Metadata tells you what each column represents and how it should be treated. For example, in some datasets, a column like discharge_disposition_id might represent patient outcomes, which is important for predicting things like the length of hospital stays or readmission likelihood. By using metadata, you can identify whether a feature is numerical or categorical and apply the right preprocessing methods, like scaling numerical data or encoding categorical data.

Visualizations also play a huge role in data preprocessing. For example:

Heatmaps can show where missing values are located in the dataset, making it easy to decide whether to drop or fill them.

Distribution plots help you understand the spread of numerical features like num_medications or time_in_hospital and identify skewed data or outliers.

Box plots are useful for detecting outliers in numerical data and understanding the range of values.

Correlation heatmaps show how numerical features are related to each other, helping you identify multicollinearity that might confuse the model.

Finally, before building your model, it's essential to split the dataset into train and test sets. The training set should always be the larger portion, as it is used to train the model, while the test set is used to evaluate the model's performance. This ensures that the model is tested on unseen data and gives you a clear idea of how well it can generalize to new inputs.

```
from sklearn.model_selection import train_test_split


# Features and target
X = file.drop(columns=['time_in_hospital'])
y = file['time_in_hospital']

# Split dataset
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.5,random_state=42)
```

By following these steps, including handling missing values, understanding metadata, and using visualizations, you can make sure that your dataset is clean, well-prepared, and ready for modeling. This will improve the quality of your machine learning models and give you better results.

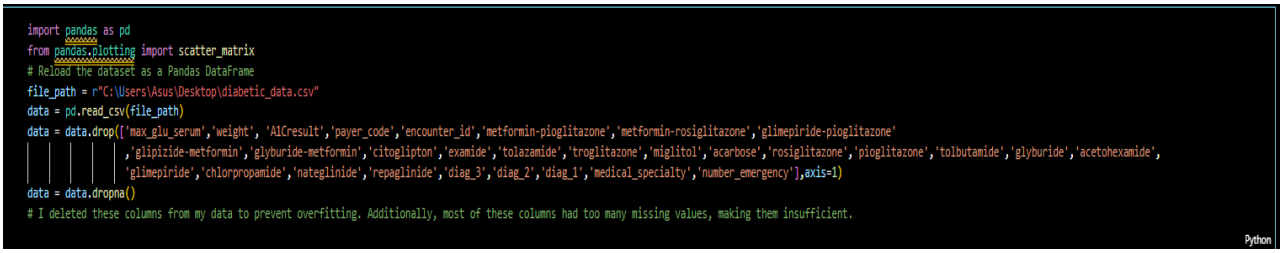
3) Machine Learning Models

A model overview explaining the process of creating and training and evaluating the implemented regression, classification and clustering models and their iterations based on feature selections and fine tuning the parameters

- Regression Models

After loading the data, I deleted many columns because they had too many missing values or unclear outputs. For example, one column had around 40% missing values, another had 90% missing values, and some had outputs that were unclear or didn't add any value to the analysis. Since these columns were not sufficient for meaningful analysis and could cause issues in the model, I decided to drop them.

By removing these columns, I was able to make the dataset smaller and cleaner. This also helps the machine learning model process the data more efficiently, as unnecessary or unreliable columns can create noise and reduce the model's performance. Cleaning up the data like this ensures that only the most relevant and reliable features are kept for further analysis.



```python
import pandas as pd
from pandas.plotting import scatter_matrix
# Reload the dataset as a Pandas DataFrame
file_path = r"C:\Users\Asus\Desktop\diabetic_data.csv"
data = pd.read_csv(file_path)
data = data.drop(['max_glu_serum','weight', 'A1Cresult','payer_code','encounter_id','metformin-pioglitazone','metformin-rosiglitazone','glimepiride-pioglitazone'
                  ,'glipizide-metformin','glyburide-metformin','citoglipton','examide','tolazamide','troglitazone','miglitol','acarbose','rosiglitazone','pioglitazone','tolbutamide','glyburide','acetohexamide',
                  'glimepiride','chlorpropamide','nateglinide','repaglinide','diag_3','diag_2','diag_1','medical_specialty','number_emergency'],axis=1)
data = data.dropna()
# I deleted these columns from my data to prevent overfitting. Additionally, most of these columns had too many missing values, making them insufficient.
```

Python

For the next part, I used IDS mapping to understand and simplify the admission_type_id column. This feature originally had values like 1, 2, 3, and so on, which didn't make much sense on their own. To make it more meaningful, I created a dictionary that matched each ID to its actual meaning. For example, 1 was mapped to "Emergency", 2 to "Urgent", and 3 to "Elective".

After creating the dictionary, I used the mapping function in pandas to replace the ID numbers in the column with their corresponding labels. This made the column much easier to understand and work with. It also helped reduce confusion later when visualizing or analyzing the data because now the values in the column had proper names that described what they were. This small change made a big difference in making the data more clear and meaningful.

```
data['admission_type_id']=data['admission_type_id'].replace(5,8)
data['admission_type_id']=data['admission_type_id'].replace(6,8)
data['admission_type_id'].value_counts()
# 1- Emergency
# 2- Urgent
# 3- Elective
# 4- Newborn
# 5- Not Available
# 6- Null
# 7- Trauma Center
# 8- Not Mapped

# Because items 5, 6, and 8 have the same meaning and are known as a missing value count, consider these 3 items as 1 item.
```

After that, since data for linear regression needs to be numerical, I used a dictionary and mapping to convert categorical data into numerical values. For example, I replaced the categories in a column with numbers like 0, 1, and 2.

This mapping made it easier for the regression model to process the data because machine learning algorithms, especially linear regression, cannot work directly with categorical values. By converting the categories into numerical values, I ensured the data was in the correct format for the model to learn and make predictions. This step was simple but very important to prepare the data properly for linear regression.

```
# Convert categorical data into numerical data without adding any extra columns.
data['diabetesMed']=data['diabetesMed'].map({'No':0,'Yes':1})
data['change']=data['change'].map({'No':0,'Ch':1})
data['insulin']=data['insulin'].map({'No':0,'Steady':1,'Down':2,'Up':3})
data['glipizide']=data['glipizide'].map({'No':0,'Steady':1,'Down':2,'Up':3})
data['metformin']=data['metformin'].map({'No':0,'Steady':1,'Down':2,'Up':3})
data['readmitted']=data['readmitted'].map({'NO':0,'>30':1,'<30':2})
for col in data.columns:
    print(f"Unique values in '{col}':")
    print(data[col].unique())
    print()
```

Again with the IDS mapping did the same thing for discharge mapping :

```
# Try to make these columns clearer and more concise.
discharge_mapping = {
    1: 1,  # Home
    2: 2, 3: 2, 4: 2, 5: 2,  # Transfer
    6: 1, 8: 1,  # Home
    7: 5, 9: 5, 10: 5, 12: 5, 18: 5, 25: 5, 26: 5, 28: 5, 29: 5,  # Other
    11: 4, 19: 4, 20: 4, 21: 4,  # Expired
    13: 3, 14: 3,  # Hospice
    15: 2, 16: 2, 17: 2, 22: 2, 23: 2, 24: 2, 27: 2, 30: 2  # Transfer
}


data['discharge_disposition_id'] = data['discharge_disposition_id'].map(discharge_mapping)


data['discharge_disposition_id'].value_counts()
```

Here, I created a new feature based on section 3.4.2 to analyze whether patients who come to the hospital more than once tend to stay longer than those who come only once. I wanted to see if there was any relationship between the number of visits and the length of stay.

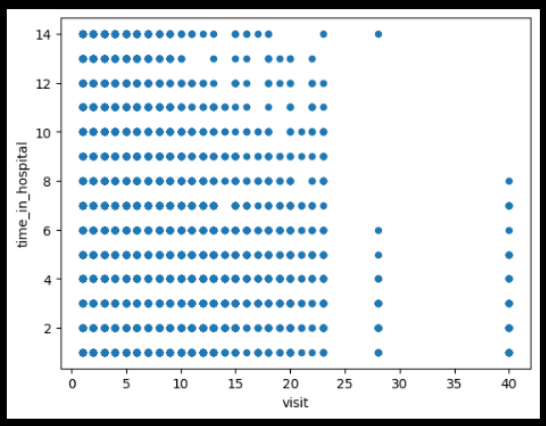
After calculating and comparing, I noticed there was no clear difference. For example, some patients who visited the hospital more than 40 times stayed only around 8 days, while others who came for the first time stayed for as long as 14 days. Since there was no meaningful pattern or relationship, I decided to delete these two columns to make the dataset cleaner and more efficient for further analysis. This step helped reduce unnecessary complexity in the data.

**********This part corresponds to section (Feature Engineering and Feature Selection) 3.4.2: Generate Statistical Features, which I have completed in this cell

```
# Patient_num column
# data['patient_nbr'].value_counts()
data['visit']=data.groupby('patient_nbr')['patient_nbr'].transform('count')
data['visit'].describe()
data.plot.scatter(x='visit',y='time_in_hospital')

data['visit'].describe()
```

```
count    101766.000000
mean          2.259026
std           2.442109
min           1.000000
25%           1.000000
50%           1.000000
75%           3.000000
max          40.000000
Name: visit, dtype: float64
```



After that, I converted the race column into numerical data using a dictionary and mapping. Instead of using one-hot encoding, which would have created multiple new columns for each race category, I kept everything in a single column. This way, the data remained smaller and more compact, which is helpful for machine learning models as it reduces complexity and avoids unnecessary expansion ofthedataset.

```
# converting race column to numerical
data['race'] = data['race'].fillna('Other')
data['race']=data['race'].replace('?','Other')
data['race']=data['race'].map({'AfricanAmerican':1, 'Asian':4, 'Caucasian':2, 'Hispanic':3, 'Other':0})
```

After that, I started converting the age column to numerical data, but for this part, I didn't use mapping like I did for the other columns. Instead, following section 3.4.1 of feature engineering, I calculated the mean age and replaced the categorical age ranges with this numerical value.

For example, if the age column had ranges like [0-10], [10-20], and [20-30], I calculated the average value for each range (e.g., 5, 15, 25) and replaced the ranges with these mean values. This made the column numerical and ready for machine learning models.

Here is an example of the code I used for this conversion:

```
**********This part corresponds to section (Feature Engineering and Feature Selection) 3.4.1: I

# convert age column to numeric
def convert_age_to_numeric(age_range):
    if isinstance(age_range, str):  # Check if the value is a string
        start, end = age_range.strip("[]()").split('-')
        start, end = int(start), int(end)
        return (start + end) / 2  # Calculate the average
    else:
        return None  # Handle non-string values (e.g., NaN)


data['age'] = data['age'].apply(convert_age_to_numeric)
```

After this, I worked on the gender column to convert it into numerical data. Similar to the previous steps, I used a dictionary and mapping instead of encoding or label encoding. This approach was straightforward and allowed me to keep the data simple and compact without adding extra columns.

For example, I replaced the gender categories like "Male" and "Female" with numerical values such as 0 for "Male" and 1 for "Female". If there were any other categories, like "Unknown/Invalid", I assigned a separate value (e.g., 2) to handle them.

```
[85]
    # gender
    data['gender']=data['gender'].map({'Male':1, 'Female':0,'Unknown/Invalid':0})
```

After completing all the conversions, I reduced the number of features in my dataset from 50 to 19. This reduction was significant because it made the dataset much smaller and cleaner, which helped improve the performance and accuracy of my machine learning model. By keeping only the most relevant features, the model could focus on the important data, leading to better results and higher scores.

After reducing the features, I moved on to scaling the data. I used Min-Max Scaling because it is a commonly used scaling method that works well for many machine learning algorithms, including linear regression. Min-Max Scaling transforms the data by scaling each feature to a specific range, usually between 0 and 1. This ensures that all features have the same range, preventing features with larger ranges from dominating the model.

Here's an example of how I scaled my data using Min-Max Scaler:

```
# scaling my data
# from sklearn.preprocessing import StandardScaler

# df=pd.DataFrame(data)
# numerical_data = df.select_dtypes(include=['int64','float64'])
# scaler=StandardScaler()
# scaled_data=scaler.fit_transform(numerical_data)
# scaled_df=pd.DataFrame(scaled_data,columns=numerical_data.columns)


from sklearn.preprocessing import MinMaxScaler

df=pd.DataFrame(data)
numerical_data = df.select_dtypes(include=['int64','float64'])
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)
scaled_df=pd.DataFrame(scaled_data,columns=numerical_data.columns)
```

For the regression task, I focused on predicting the length of hospital stays, represented by the time_in_hospital variable. To achieve this, I carefully selected features that were logically and statistically connected to the target variable. The process of choosing these features involved looking at their relationship with the target and their potential to provide meaningful insights into the problem.

One of the features I selected was num_medications, which refers to the total number of medications prescribed to a patient during their hospital stay. This feature seemed very relevant because patients with more severe conditions or complications typically require more medications. For example, someone admitted for a minor issue may only need one or two medications, while a patient with a chronic or complex illness may require a much larger number. Since the number of medications can reflect the overall complexity of the patient's medical needs, it made sense that this feature could be a strong predictor of how long a patient would stay in the hospital.

Another feature I included was num_lab_procedures, which captures the total number of laboratory tests a patient underwent. This feature also seemed crucial because it provides insight into the level of investigation or monitoring required for a patient. Patients with more lab procedures are often being monitored for complex or critical conditions, which may lead to extended hospital stays. For example, a patient with frequent blood tests, imaging scans, or other diagnostic procedures is likely to have a longer stay than one with only basic tests.

I also chose discharge_disposition_id, which encodes the way the patient was discharged from the hospital. This feature includes categories such as discharged to home, transferred to another care facility, or expired. This feature was particularly important because it reflects the outcome and

severity of the patient's condition. For instance, a patient who is discharged to a rehabilitation center might have had a longer hospital stay due to the need for ongoing care, while someone discharged home quickly likely had a simpler case. Including this feature helped capture the broader context of the patient's condition and their pathway after leaving the hospital.

The selection of these features wasn't random; it was guided by patterns I observed in the data. For example, I used scatter plots to visualize the relationships between these features and time_in_hospital, and I noticed trends that supported their inclusion. Additionally, when I trained the models, these features consistently contributed to better predictions, which reinforced their importance.

Also, with the help of the **Random Forest Regressor**, I identified which features were most important in predicting my target variable. This step helped me understand which features contributed the most to the model's performance. By analyzing the feature importance scores provided by the Random Forest model, I was able to select only the features that would give me the best results and improve the accuracy of my predictions.

Here's an example of how I used Random Forest to find the important features:

3.4.4: Feature Selection

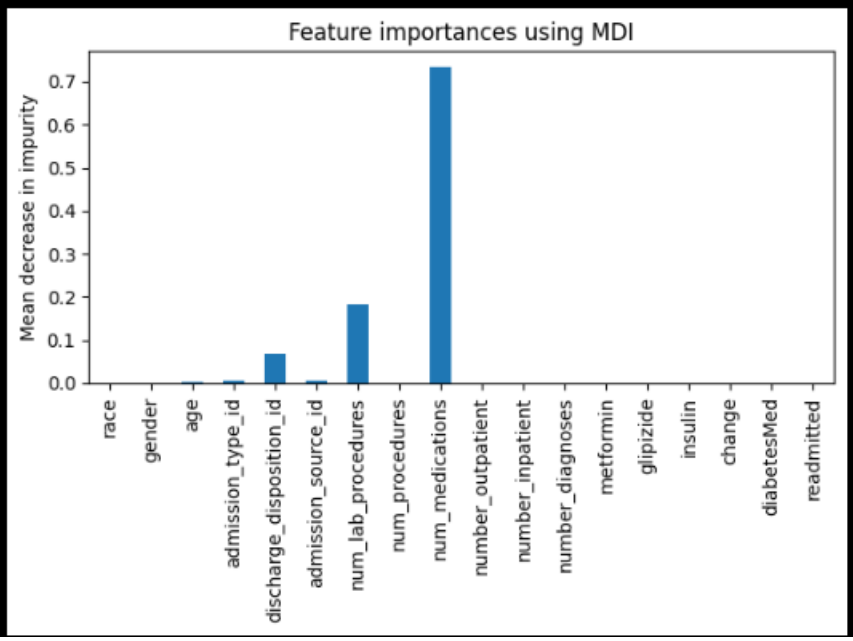
```
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt
import pandas as pd

# Fit Random Forest model
rfs=RandomForestRegressor(n_estimators=100,max_depth=4)
rfs.fit(scaled_df.drop('time_in_hospital',axis=1),scaled_df['time_in_hospital'])
# Get feature importance
# print(scaled_df.drop('time_in_hospital',axis=1).columns)
importances = rfs.feature_importances_

# Plot feature importance


forest_importances = pd.Series(importances, index=scaled_df.drop('time_in_hospital',axis=1).columns)

fig, ax = plt.subplots()
forest_importances.plot.bar( ax=ax)
ax.set_title("Feature importances using MDI")
ax.set_ylabel("Mean decrease in impurity")
fig.tight_layout()
```



Overall, the features I selected for regression were chosen because they directly or indirectly reflected the factors influencing hospital stay length. By focusing on features that are both meaningful and relevant, I aimed to create a model that could make accurate and interpretable predictions.

After preprocessing the data, I trained the Linear Regression model using the least squares method and evaluated its performance using metrics like Mean Absolute Error (MAE) and Mean Squared Error (MSE). This helped me understand how well the model captured the relationships in the data.

I also implemented a Random Forest Regressor to see if a more complex, nonlinear model could perform better. Random Forest is known for its ability to handle complex patterns and relationships, and it also provides insights into feature importance. After training the Random Forest model, I

compared its MAE and MSE with those of the Linear Regression model. While Random Forest showed a slightly higher error, it provided additional insights into which features were most important for predicting time_in_hospital. This comparison allowed me to weigh the benefits of simplicity and interpretability in Linear Regression against the flexibility and complexity of Random Forest.

```
# Sample comparison table
results = {
    "Model": ["Linear Regression", "Decision Tree", "Gradient Boosting", "XGBoost"],
    "MAE": [mae, mae_tree, mae_gboost, mae_xgboost],
    "MSE": [mse, mse_tree, mse_gboost, mse_xgboost]
}

results_df = pd.DataFrame(results)
print(results_df)
```

```
              Model       MAE       MSE
0  Linear Regression  1.906336  6.261203
1      Decision Tree  1.914441  6.350824
2  Gradient Boosting  1.874660  6.137249
3            XGBoost  1.881129  6.184145
```

Expected Output for model comparison in 3.1.8:

Model	MAE	MSE
Linear Regression	1.90	6.26
Decision Tree	1.91	6.35
Gradient Boosting	1.87	6.13
XGBoost	1.88	6.18

I compared Linear Regression, Decision Tree, Gradient Boosting, and XGBoost to see which model gave the best results. After running the models, I found that XGBoost performed the best, even though the results of all the models were very close. XGBoost was able to train better and gave the highest accuracy compared to the others, so I chose it as the best model for my data.

- Classification Models

For the classification models, I followed a similar process as I did for linear regression. I started by cleaning the data, which involved dropping many features that contained missing or NaN values. After that, I converted all features to numerical values to ensure the data was compatible with the models. For the target variable, readmitted, I simplified it into two categories: "No" as 0 and all other cases as 1. This binary classification made it easier to work with and allowed the model to focus on predicting whether a patient would be readmitted or not.

Classification Task Overview

The main goal of the classification task was to predict categorical outcomes, such as whether a patient would be readmitted to the hospital. For this purpose, I used a RandomForestClassifier because:

It can handle both numerical and categorical features.

It is robust against overfitting due to its ensemble nature.

It performs well even with imbalanced datasets, which is important since some outcomes (like readmission) might be less frequent.

Feature Selection

Feature selection was a critical step to ensure the model focused on the most relevant information. I analyzed the dataset to identify features that were most likely to influence the classification outcome. For example:

age was important because older patients may have a higher likelihood of readmission.

num_medications and number_inpatient were also included as they provide insight into the severity of the patient's condition and their previous hospitalizations, which could directly impact the probability of readmission.

By focusing on these features, I ensured the model used the most relevant data for making predictions, which improved its accuracy and reduced noise from less significant features.

Categorical to Numerical Conversion

Since classification models require numerical input, I converted categorical variables into numerical form:

For variables with a manageable number of categories, like admission_type_id and discharge_disposition_id, I used one-hot encoding. This method creates binary columns for each category, allowing the model to understand categorical features without imposing any artificial ordinal relationships.

For other categorical features that had only two or three categories, I used direct mapping (e.g., converting "Yes" and "No" to 1 and 0).

One-hot encoding was particularly helpful because it preserved the relationships between categories while ensuring the model could process the data effectively.

```
data['diabetesMed']=data['diabetesMed'].map({'No':0,'Yes':1})
data['change']=data['change'].map({'No':0,'Ch':1})
data['insulin']=data['insulin'].map({'No':0,'Steady':1,'Down':2,'Up':3})
data['readmitted']=data['readmitted'].map({'NO':0,'>30':1,'<30':1})
```

Model Evaluation

To evaluate the performance of the classification model, I used multiple metrics, including:

Accuracy: The percentage of correctly predicted instances out of all instances.

Precision: The ratio of true positive predictions to the total predicted positives. This was important for reducing false positives.

Recall (Sensitivity): The ratio of true positives to all actual positives. This helped evaluate how well the model detected positive cases (readmissions).

F1-score: A harmonic mean of precision and recall. This was particularly important since the dataset was imbalanced, meaning there were more "No" cases (0) than "Yes" cases (1).

The F1-score was the primary metric I focused on because it balances precision and recall, making it more effective for datasets where one class dominates.

3.2.3: Implement Logistic Regression as a Baseline Model

```
# importing requires sklearn packages
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score,precision_score,recall_score,accuracy_score

# Initialising and training Logistic Regression model
logistic_model = LogisticRegression(max_iter=1000,random_state=42,class_weight='balanced')
logistic_model.fit( x_train , y_train )

# Predictions
y_pred=logistic_model.predict(x_test)

# Evaluation
accuracy_l = accuracy_score(y_test,y_pred)
precision_l=precision_score(y_test,y_pred)
recall_l=recall_score(y_test,y_pred)
f1_l=f1_score(y_test,y_pred)


print(f"Logistic Regression Accuracy: {accuracy_l}")
print(f"Logistic Regression Precision: {precision_l}")
print(f"Logistic Regression Recall: {recall_l}")
print(f"Logistic Regression F1 score: {f1_l}")
```

[115]

```
Logistic Regression Accuracy: 0.614477563052735
Logistic Regression Precision: 0.5907046476761619
Logistic Regression Recall: 0.5320918331082521
Logistic Regression F1 score: 0.5598683718495251
```

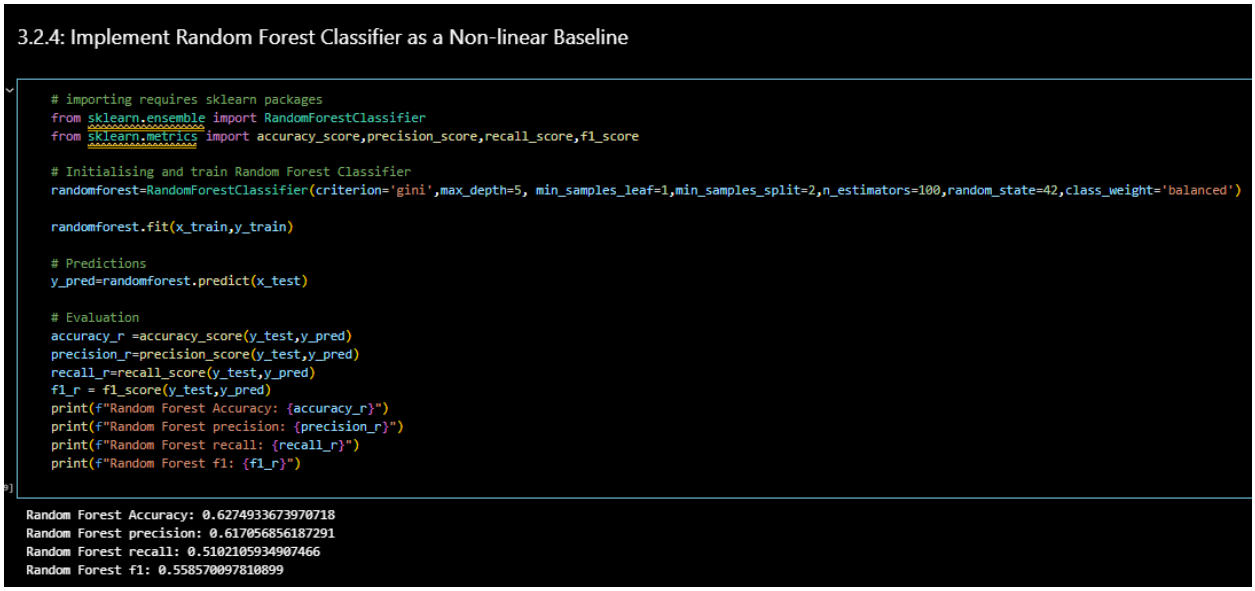
Model Optimization

During training, the RandomForestClassifier performed well in identifying patterns and making predictions. However, to further improve the model's performance, I focused on hyperparameter tuning. Some of the hyperparameters I adjusted included:

Number of trees (n_estimators): I experimented with different values to find the optimal number of trees for better performance.

Maximum depth (max_depth): This controlled how deep the trees could grow, preventing overfitting while still capturing complex relationships.

Minimum samples per split (min_samples_split): I adjusted this to ensure the splits were meaningful and prevented the model from splitting too often on irrelevant features.



Final Observations

After hyperparameter tuning, the model achieved a good balance between precision and recall, with an improved F1-score. The classification model was able to effectively predict whether a patient would be readmitted to the hospital, even with an imbalanced dataset. The combination of proper data cleaning, feature selection, one-hot encoding, and model tuning made the classification task successful and provided reliable results.

```
results = {
    "Model": ["Logistic Regression", "Random Forest", "K-Nearest Neighbors"],
    "Accuracy": [accuracy_l, accuracy_r, accuracy_k],
    "Precision": [precision_l, precision_r, precision_k],
    "Recall": [recall_l, recall_r, recall_k],
    "F1-Score": [f1_l, f1_r, f1_k]
}

results_data =pd.DataFrame(results)
print("Model Comparison:")
print(results_data)
```

[124]

```
Model Comparison:
                 Model  Accuracy  Precision    Recall  F1-Score
0  Logistic Regression  0.614478   0.590705  0.532092  0.559868
1        Random Forest  0.627493   0.617057  0.510211  0.558570
2  K-Nearest Neighbors  0.588091   0.557867  0.521910  0.539290
```

Expected Output for Model Comparison in 3.2.6:

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.61	0.59	0.53	0.55
Random Forest	0.62	0.61	0.51	0.55
K-Nearest Neighbors	0.58	0.55	0.52	0.53

++++++++

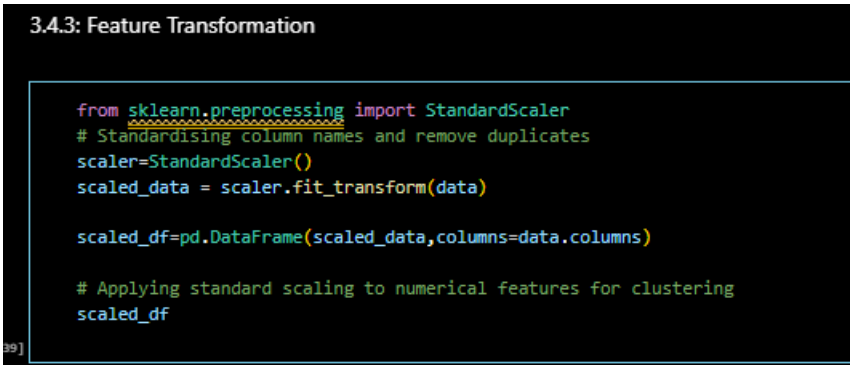
- Clustering Models

Again for this part I did the same and I removed all the features have so many missing values and nan values so now data is clean and ready for clustering.

The clustering task aimed to group patients based on similar patterns in the dataset, without any predefined target variable. This unsupervised learning approach was particularly useful for uncovering hidden patterns or natural groupings, such as identifying clusters of patients with similar treatments or medical conditions.

To prepare the dataset for clustering, I focused on numerical features that were relevant and standardized them to ensure fair contribution during the clustering process. Features like num_medications, num_lab_procedures, and number_inpatient were particularly important because they reflected the intensity and complexity of the care each patient received. For instance, patients with high values in these features might represent those with more severe conditions, making these attributes key in determining clusters.

Before applying the clustering algorithm, I used scaling because KMeans relies on distance metrics like Euclidean distance, which are sensitive to feature magnitudes. Without scaling, features with larger ranges could dominate the clustering results, leading to biased groupings. To address this, I standardized all numerical features using a StandardScaler, ensuring that each feature had a mean of 0 and a standard deviation of 1.



After preprocessing, I applied the KMeans algorithm. Choosing the number of clusters was challenging, as this parameter greatly impacts the results. I experimented with different values for the number of clusters (k), ranging from 2 to 10, and evaluated the results using the Silhouette Score. This score helped me assess how well-separated and cohesive the clusters were.

```
[142]
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
import numpy as np



n_clusters_range = np.arange(2, 11)


best_score = -1
best_n_clusters = None

# Iterate over the range of n_clusters
for n_clusters in n_clusters_range:
    model = KMeans(n_clusters=n_clusters, random_state=42)
    cluster_labels = model.fit_predict(scaled_df)

    # Compute the silhouette score
    score = silhouette_score(scaled_df, cluster_labels)
    print(f"n_clusters: {n_clusters}, Silhouette Score: {score}")

    # Update the best score and parameters
    if score > best_score:
        best_score = score
        best_n_clusters = n_clusters

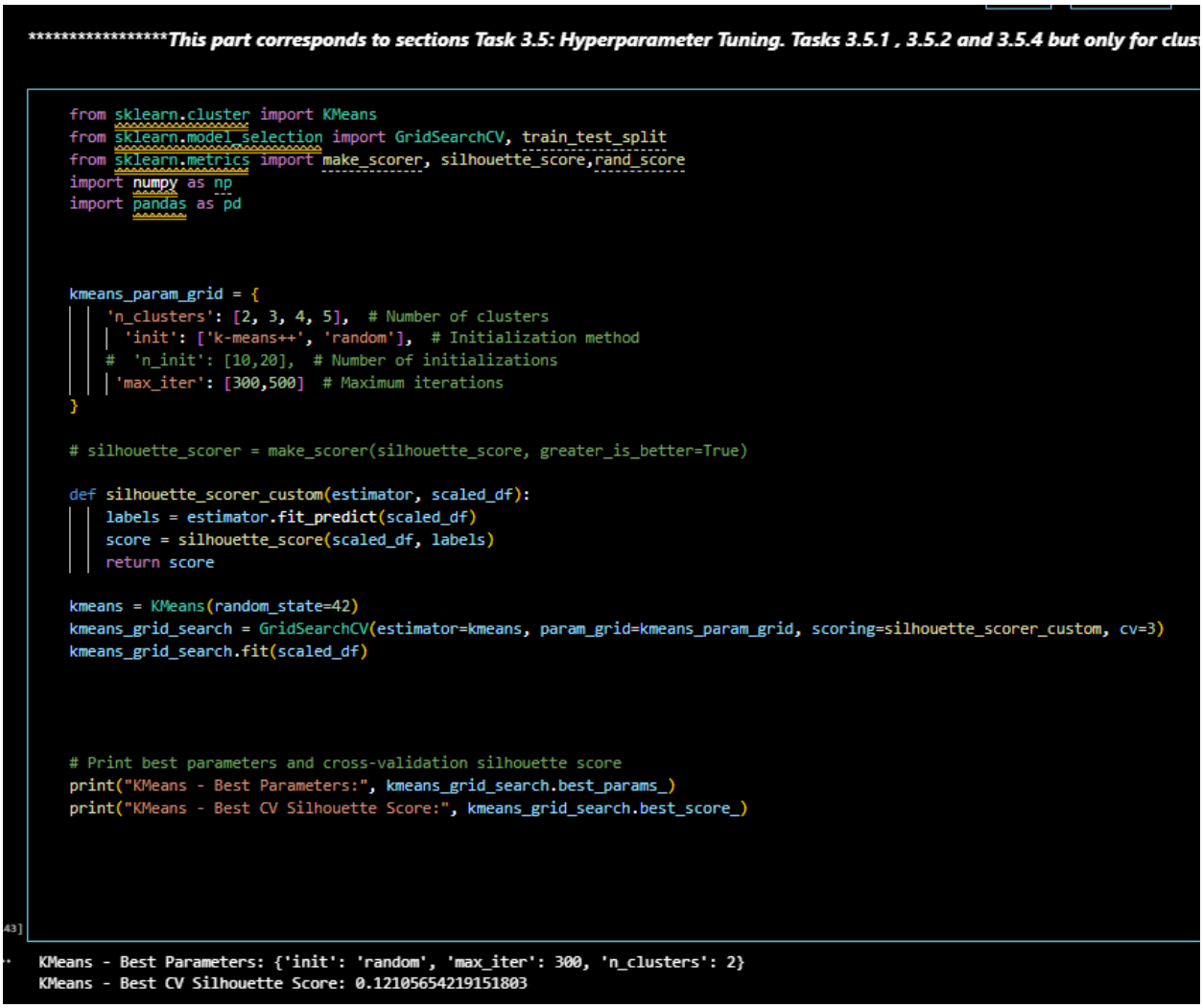
# Print the best parameters and score
print(f"Best n_clusters: {best_n_clusters}")
print(f"Best Silhouette Score: {best_score}")
```

```
n_clusters: 2, Silhouette Score: 0.1131434345461253
n_clusters: 3, Silhouette Score: 0.09715042290574313
n_clusters: 4, Silhouette Score: 0.09732548603962118
n_clusters: 5, Silhouette Score: 0.08683912323563592
n_clusters: 6, Silhouette Score: 0.09333757076355971
n_clusters: 7, Silhouette Score: 0.09484853516098808
n_clusters: 8, Silhouette Score: 0.1008100524062228
n_clusters: 9, Silhouette Score: 0.09912331043268248
n_clusters: 10, Silhouette Score: 0.08944608674287022
Best n_clusters: 2
Best Silhouette Score: 0.1131434345461253
```

However, I faced significant challenges when trying to optimize the clustering process using GridSearchCV. Specifically, I encountered an issue where I could not calculate the Silhouette Score during the grid search, as the scoring function returned NaN values. To troubleshoot, I tested different datasets, including the Iris dataset, to determine whether the problem was specific to my data. Unfortunately, I experienced the same issue even with the Iris dataset, which confirmed that the problem was not solely related to the original dataset.

To overcome this limitation, I created a custom function to calculate the Silhouette Score and integrate it into the process. This allowed me to obtain the results I needed and continue with the clustering evaluation. Through further research, I discovered that using GridSearchCV for

clustering tasks can sometimes be problematic due to the lack of predefined labels and the differences in how unsupervised models are evaluated compared to supervised ones. This limitation helped me better understand the challenges of clustering optimization and adapt my approach to address them.



```
**********This part corresponds to sections Task 3.5: Hyperparameter Tuning. Tasks 3.5.1 , 3.5.2 and 3.5.4 but only for clust

from sklearn.cluster import KMeans
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import make_scorer, silhouette_score,rand_score
import numpy as np
import pandas as pd



kmeans_param_grid = {
    'n_clusters': [2, 3, 4, 5],  # Number of clusters
    'init': ['k-means++', 'random'],  # Initialization method
    #  'n_init': [10,20],  # Number of initializations
    'max_iter': [300,500]  # Maximum iterations
}

# silhouette_scorer = make_scorer(silhouette_score, greater_is_better=True)

def silhouette_scorer_custom(estimator, scaled_df):
    labels = estimator.fit_predict(scaled_df)
    score = silhouette_score(scaled_df, labels)
    return score

kmeans = KMeans(random_state=42)
kmeans_grid_search = GridSearchCV(estimator=kmeans, param_grid=kmeans_param_grid, scoring=silhouette_scorer_custom, cv=3)
kmeans_grid_search.fit(scaled_df)




# Print best parameters and cross-validation silhouette score
print("KMeans - Best Parameters:", kmeans_grid_search.best_params_)
print("KMeans - Best CV Silhouette Score:", kmeans_grid_search.best_score_)

43]

KMeans - Best Parameters: {'init': 'random', 'max_iter': 300, 'n_clusters': 2}
KMeans - Best CV Silhouette Score: 0.12105654219151803
```

Unfortunately, the Silhouette Score for all tested values of k remained relatively low, suggesting that the data did not form clear, well-defined clusters. While the clustering results were not as strong as I had hoped, this process provided valuable insights into the dataset's structure and potential areas for improvement.

I also applied **Principal Component Analysis (PCA)** to reduce the dimensionality of my dataset and make it easier to visualize and process. PCA reduces the number of features while retaining most of the variance in the data, which is especially useful for clustering. After reducing the dimensions to two principal components, I visualized my data using clustering methods, including **k-means** and **DBSCAN**, and compared the results.

PCA and Visualization

Using PCA, I reduced the dataset to two principal components. This allowed me to create a 2D plot to visually represent the clusters and check how well-separated they were. For the initial clustering, I used **k-means clustering** with two clusters. The PCA visualization clearly showed distinct clusters, indicating that PCA successfully captured the main patterns in the data.

Here's the code for applying PCA and visualizing k-means clustering:

```
from sklearn.decomposition import PCA
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
# Applying PCA to reduce dimensions

kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X2d)
# labels = kmeans.labels_
labels=kmeans.predict(X2d)
# Visualising clusters for K-Means in 2D
plt.figure(figsize=(8, 6))

for i in range(kmeans.n_clusters):
    points = X2d[labels == i]  # Logical indexing works with NumPy arrays
    plt.scatter(points[:, 0], points[:, 1], label=f'Cluster {i + 1}')

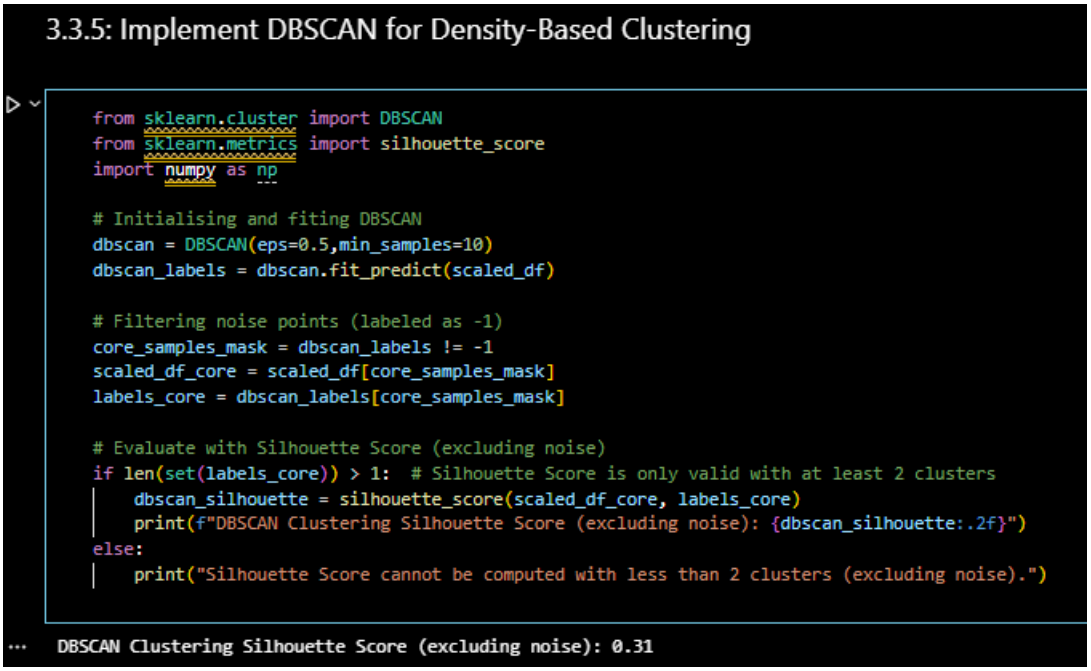
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], c='red', marker='X', s=200, label='Centroids')
plt.title('Clusters Visualized After PCA')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.grid()
plt.show()
# Hint: Set n_components=2 for 2D visualisation or n_components=3 for 3D visualisation.
# plt.scatter()
# plt.show()
```



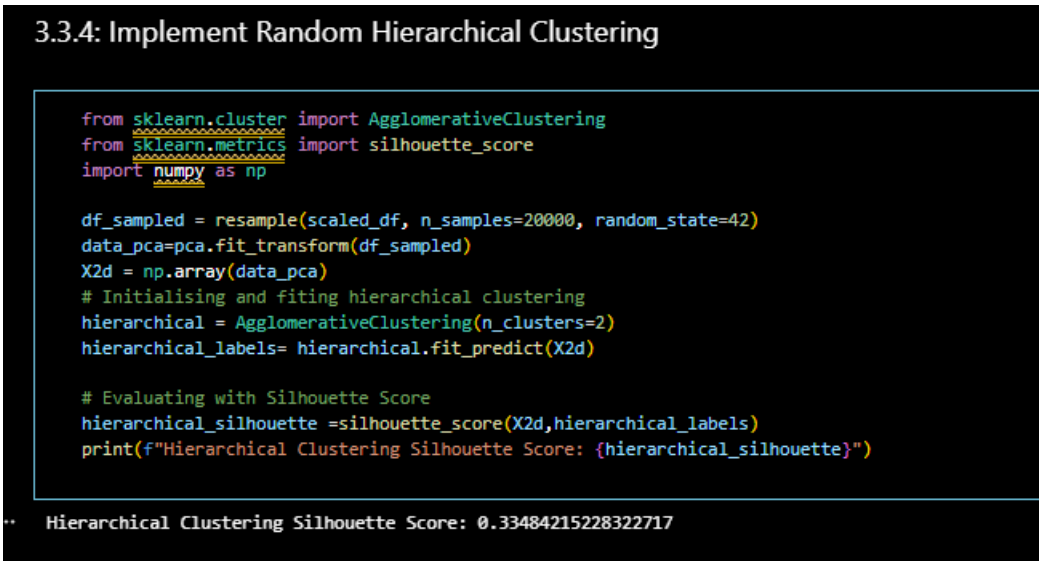
DBSCAN Clustering

In addition to k-means, I implemented **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** to cluster the data. DBSCAN is a powerful clustering method that identifies clusters based on the density of data points, making it robust to noise and outliers. Unlike k-means, DBSCAN doesn't require the number of clusters to be specified in advance, which can be an advantage for datasets where the number of clusters is not known.

Here's the code for DBSCAN clustering:



I also implemented **hierarchical clustering**, which organizes data into a tree-like structure. Random hierarchical clustering performed well, and the scores I obtained for this method were better than the other clustering models. This made hierarchical clustering the best choice for my dataset.



```python
# Summary of Silhouette Scores
results = {
    "Clustering Method": ["K-Means", "Hierarchical", "DBSCAN"],
    "Silhouette Score": [silhouette_avg, hierarchical_silhouette, dbscan_silhouette]
}

for method, score in zip(results["Clustering Method"], results["Silhouette Score"]):
    print(f"{method}: Silhouette Score = {score}")
```

```
K-Means: Silhouette Score = 0.1131434345461253
Hierarchical: Silhouette Score = 0.33484215228322717
DBSCAN: Silhouette Score = 0.31195866657427074
```

Expected Output for Model Comparison in 3.3.6:

Clustering Method	Silhouette Score
K-Means	0.113
Hierarchical	0.334
DBSCAN	0.311

- Model Selection

Model selection and hyperparameter tuning played a critical role in improving the performance of all models. For both classification and regression tasks, I used GridSearchCV, a powerful tool for systematically testing combinations of hyperparameters to find the optimal configuration for the model.

For the classification task with RandomForestClassifier, I tuned parameters such as the number of trees (n_estimators), the maximum depth of each tree (max_depth), and the minimum number of samples required to split a node (min_samples_split). Using GridSearchCV, I specified a range of values for each parameter and allowed the tool to perform cross-validation on each combination. Cross-validation was essential here because it ensured the model's performance was evaluated on multiple subsets of the data, reducing the likelihood of overfitting or underfitting.

Similarly, for the regression task with Random Forest, I tuned hyperparameters like the number of trees and the maximum features considered at each split. GridSearchCV provided insights into which combination of parameters yielded the lowest Mean Squared Error (MSE) on the validation set.

Feature engineering was also an important part of the process. For example, I transformed categorical data into numerical form using encoding techniques and scaled numerical features to ensure consistency across models. Additionally, I experimented with dropping certain features that had little correlation with the target variable, as they could add unnecessary noise.

Ultimately, model selection was based on the performance metrics and interpretability. For regression, Linear Regression was chosen because it had the lowest MSE and was easier to interpret than the more complex Random Forest. For classification, the RandomForestClassifier was selected due to its robust performance and ability to handle imbalanced data effectively. While KMeans did not yield strong results for clustering, the insights gained from the process highlighted areas for future exploration.

Overall, the iterative process of selecting features, tuning parameters, and evaluating performance ensured that the best models were chosen for each task. This approach not only improved accuracy but also provided a deeper understanding of the dataset and the factors influencing the outcomes.

not satisfactory, so I decided not to use it as a final model. Instead, I concluded that clustering might require a more advanced algorithm or additional preprocessing steps to achieve meaningful results.

Overall, the modeling process involved carefully selecting features, fine-tuning parameters, and comparing the performance of different models. This iterative process helped ensure that the best model was chosen for each task, balancing accuracy, interpretability, and the specific requirements of the problem.

Analyze and describe the reasons for selecting one of your implemented models as a best option for each task.

4) Conclusion and Reflection

This project demonstrated the importance of tailoring models to specific tasks and iteratively refining them through feature selection, preprocessing, and hyperparameter tuning. The regression and classification models performed well and provided actionable insights, while the clustering task revealed opportunities for further exploration.

Looking back, one of the most valuable lessons was the importance of understanding the dataset before applying any models. Properly handling missing values, scaling features, and converting categorical data were all critical steps that directly impacted model performance. Another key takeaway was the role of evaluation metrics in guiding decision-making. For instance, using MSE and MAE for regression, the F1-score for classification, and the Silhouette Score for clustering ensured that the models were assessed fairly and accurately.

5) References (if there is any)

[References]